DOCUMENT DE CADRAGE

Projet Agent Autonome Gemma 4

Contexte du Projet : Compétition Kaggle "The Gemma 4 Developer Agent"

Objectif : Développer un agent de génie logiciel autonome fonctionnant hors ligne sur du matériel grand public (16 Go RAM).

Enjeux : Cagnotte totale de 100 000 $. Date limite de soumission : 25 novembre 2026.

Lead Technique : Max_Adis (Maker, Développeur & Ingénieur Indépendant)

1. Le Terrain (Périmètre Technique)

- **Modèle de Base :** Gemma 4 (versions 2B ou 7B quantifiées en 4-bit, format GGUF, pour garantir une inférence fluide sur 16 Go de RAM).
- **Environnement d'Exécution :** Inférence locale via WSL2, Ollama ou llama.cpp. L'agent ne doit nécessiter aucune API Cloud payante.
- **Paradigme de l'Agent :** Implémentation d'une boucle ReAct (Reason + Act) via un script Python. L'agent doit être capable de planifier ses actions, d'exécuter des requêtes, de lire les résultats du terminal et de s'auto-corriger.
- **Stratégie d'Optimisation :** Utilisation de l'approche LoRA / QLoRA (Low-Rank Adaptation) via la bibliothèque peft. L'adaptation des poids se fera sans modifier le modèle de base, permettant un apprentissage rapide et économe en ressources.

2. L'Arsenal (Outils Connectés à l'Agent)

L'agent interagira avec l'environnement de développement via un système de "Function Calling". Ces outils seront des scripts Python que l'IA pourra invoquer de manière autonome :

- **Exploration :** `lister_arborescence(chemin)` pour scanner le dépôt Git et comprendre la structure du projet.
- **Analyse :** `lire_fichier(chemin, ligne_debut, ligne_fin)` pour isoler le code source ciblé sans saturer la fenêtre de contexte (la mémoire à court terme de l'IA).
- **Action :** `appliquer_patch(chemin, ancien_texte, nouveau_texte)` pour injecter les correctifs dans les fichiers locaux.

- **Validation :** `executer_terminal(commande)` pour lancer des commandes Bash de base, exécuter des linters ou lancer les scripts de tests unitaires locaux (ex: pytest).

3. Planning d'Attaque (Roadmap du Projet)

Phase 1 : Preuve de Concept et Environnement (Fin Septembre)

- Télécharger les poids quantifiés de Gemma 4.
- Valider l'exécution du modèle en local.
- Coder le script Python minimal instanciant la boucle ReAct (Pensée -> Action -> Observation).

Phase 2 : Intégration des Outils Locaux (Début Octobre)

- Connecter le script Python au module subprocess de l'OS.
- Créer un environnement de test local (un "dossier_cobaye" avec des erreurs de code basiques) pour valider la capacité de l'agent à lire et écrire.

Phase 3 : Entraînement et LoRA (Mi-Octobre à Début Novembre)

- Analyser le dataset d'évaluation fourni par Kaggle.
- Déporter les calculs lourds d'entraînement LoRA sur les notebooks gratuits de Kaggle ou Google Colab (utilisant leurs GPU).
- Rapatrier les poids LoRA générés sur le PC local.

Phase 4 : Tests Finaux et Soumission (Novembre)

- Fusionner les poids LoRA avec le modèle local.
- Soumettre la solution finale sur la plateforme avant la date butoir du 25 novembre.

Quel moteur vas-tu utiliser ce soir pour lancer la toute première instance du modèle sur ta machine : Ollama, ou directement un script Python via HuggingFace ?